

****Contribuer à un Projet Open Source****

Partie 1 : Les fondamentaux

:::

Session 6 – Cours M1 Introduction aux Logiciels Libres

****Objectifs de la séance****

****Ce que vous saurez faire****

À la fin de cette séance, vous serez capables de :

- **Évaluer** un projet avant d'y contribuer
- **Identifier** les différents types de contributions possibles
- **Naviguer** dans la structure d'un projet open source
- **Utiliser** le workflow fork → branch → PR

****Quelques précisions importantes****

Avant de commencer, gardez en tête :

1. **Il n'y a pas que GitHub** – D'autres forges existent
2. **Il n'y a pas que Git** – D'autres systèmes de versionnement existent
3. **Tous les projets sont différents** – Adaptez-vous à chaque contexte

Cette séance utilise GitHub et Git comme exemples, car ils dominent l'écosystème actuel.

****Les forges logicielles****

Une **forge** est une plateforme collaborative pour le développement logiciel.

Principales forges :

Forge	Particularité
GitHub	La plus populaire, rachetée par Microsoft (2018)
GitLab	Open source, auto-hébergeable, CI/CD intégré
Codeberg	Non-profit, basée en Allemagne, utilise Forgejo
SourceHut	Minimaliste, workflow par email
Bitbucket	Intégration Atlassian (Jira, Confluence)
Gitea/Forgejo	Léger, auto-hébergeable

****Fonctionnalités d'une forge****

Gestion du code

- Hébergement de dépôts (Git le plus souvent, mais aussi autres)
- Branches et tags
- Historique et blame
- Comparaison de versions
- Releases

Collaboration

- Issues / bug tracker
- Pull/Merge requests
- Code review
- Wiki / documentation

Automatisation : CI/CD, tests automatiques, déploiement

Social : profils, stars/likes, followers, activité

****Au-delà de Git****

Git domine aujourd'hui, mais d'autres systèmes existent :

Système	Caractéristique
Mercurial (hg)	Similaire à Git, utilisé par certains projets (ex: Facebook)
Subversion (SVN)	Centralisé, encore utilisé dans certaines entreprises
Fossil	Inclut wiki, tickets, forum (utilisé par SQLite)
Pijul	Basé sur la théorie des patches, gestion des conflits innovante
Jujutsu (jj)	Compatible Git, undo/redo natif, développé par Google

Le workflow (fork → PR) reste conceptuellement similaire, seules les commandes changent.

****Chaque projet est unique******Ce qui varie :**

- Taille (1 personne → milliers)
- Gouvernance (BDFL, comité, fondation)
- Outils (GitHub, mailing lists, IRC)
- Process (PRs, patches par email)
- Standards de code
- Langue de communication

Conséquence :

- **Toujours** lire la documentation
- **Observer** avant de contribuer
- **S'adapter** aux pratiques du projet
- Ne pas imposer ses habitudes

■ Un projet Linux kernel ≠ un projet npm de 100 lignes

****Partie 1****

Pourquoi contribuer ?

****Motivations pour contribuer****

Pour vous

- Apprendre de nouveaux langages/outils
- Travailler avec du code de qualité
- Construire un portfolio public
- Développer votre réseau
- Résoudre un problème que vous avez

Pour votre carrière

- GitHub = "CV technique"
- Visibilité dans la communauté
- Recrutement par les mainteneurs
- Prouver ses compétences

****Recherche : Motivations des contributeurs****

Étude de Gerosa et al. (2021) – 10 catégories de motivations :

Catégorie	Description
Intrinsèque	Plaisir, challenge intellectuel
Altruisme	Aider la communauté
Réputation	Reconnaissance par les pairs
Carrière	Opportunités professionnelles
Apprentissage	Acquérir des compétences
Utilité	Résoudre son propre problème
Sociale	Appartenance à une communauté
Idéologique	Valeurs du libre
Rémunération	Payé pour contribuer
Fun	Gamification, badges

Source: [The Shifting Sands of Motivation: Revisiting What Drives Contributors in Open Source](#)

****Partie 2****

Évaluer un projet

****Avant de contribuer : Évaluez le projet****

Tous les projets ne sont pas "contributables". Vérifiez :

1. **Activité** – Le projet est-il vivant ?
2. **Communauté** – Y a-t-il d'autres contributeurs ?
3. **Accueil** – Le projet accueille-t-il les nouveaux ?
4. **Documentation** – Pouvez-vous comprendre comment contribuer ?
5. **Réactivité** – Les PRs sont-elles traitées ?

****Indicateurs de santé d'un projet****

Bons signes

- Commits récents (< 1 mois)
- Issues traitées régulièrement
- PRs mergées en temps raisonnable
- Documentation CONTRIBUTING.md
- Labels "good first issue"
- Communication active

Mauvais signes

- Dernier commit > 1 an
- Centaines d'issues non traitées
- PRs ouvertes depuis des mois
- Mainteneur unique silencieux
- Pas de documentation
- Conflits non résolus

****Métriques à vérifier****

Métrique	Où la trouver	Seuil indicatif
Dernier commit	Page principale	< 3 mois
Contributors actifs	Insights > Contributors	> 2-3 actifs récemment
Issues ouvertes	Tab Issues	Ratio fermées/ouvertes > 1
PRs ouvertes	Tab Pull requests	< 50 en attente
Temps de merge	PRs fermées	< 2 semaines médian
Stars	Page principale	Pas de seuil, contexte

****Lire le README****

Le README est la porte d'entrée du projet.

Ce qu'il devrait contenir :

- Description du projet
- Installation / Quick start
- Usage basique
- Comment contribuer (ou lien)
- Licence

Si le README est pauvre, c'est souvent un signe que le projet n'est pas prêt pour les contributions externes.

****Partie 3******Types de contributions**

****Ce n'est pas que du code !****

Contributions techniques

- Correction de bugs
- Nouvelles fonctionnalités
- Tests
- Refactoring
- Performance

Contributions non-techniques

- Documentation
- Traduction
- Design / UX
- Triage d'issues
- Réponse aux questions
- Organisation d'événements

| 80% des contributions open source ne sont pas du code.

****Par où commencer ?******Contributions idéales pour débiter :**

1. **Typos et documentation** – Risque faible, impact visible
2. **Good first issues** – Bugs simples, bien documentés
3. **Triage** – Reproduire des bugs, ajouter des infos
4. **Tests** – Augmenter la couverture
5. **Traduction** – Si vous parlez une autre langue

****Trouver des "Good First Issues"******Sur GitHub :**

- Label **good first issue**
- Label **help wanted**
- Label **beginner friendly**

Agrégateurs :

- <https://goodfirstissue.dev>
- <https://up-for-grabs.net>
- <https://github.com/MunGell/awesome-for-beginners>
- <https://firstcontributions.github.io>

****Partie 4******Structure d'un projet**

****Fichiers standards****

Fichier	Rôle
README.md	Présentation, installation, usage
LICENSE	Licence du projet
CONTRIBUTING.md	Guide de contribution
CODE_OF_CONDUCT.md	Règles de comportement
CHANGELOG.md	Historique des versions
.github/ISSUE_TEMPLATE/	Templates pour les issues
.github/PULL_REQUEST_TEMPLATE.md	Template pour les PRs

****CONTRIBUTING.md****

Ce fichier explique **comment** contribuer au projet.

Contenu typique :

- Comment reporter un bug
- Comment proposer une fonctionnalité
- Comment soumettre une PR
- Standards de code
- Processus de review
- DCO/CLA si applicable

Lisez-le AVANT de contribuer !

****CODE_OF_CONDUCT.md****

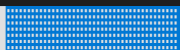
Définit les comportements acceptables dans la communauté.

Standards courants :

- Contributor Covenant (le plus répandu)
- Citizen Code of Conduct
- Ubuntu Code of Conduct

Objectif :

- Créer un environnement inclusif
- Prévenir les comportements toxiques
- Définir les recours en cas de problème

****Partie 5******Le workflow de contribution**

****Avant de coder : « réservez » l'issue****

Règle importante : Ne commencez jamais à coder sans prévenir !

Pourquoi ?

- Éviter le travail en double
- Montrer votre intérêt
- Obtenir des conseils avant de commencer

Comment ?

1. Vérifiez que personne n'y travaille déjà (commentaires, assignee)
2. Commentez : « I'd like to work on this, is it still available? »
3. Attendez confirmation (ou commencez si pas de réponse après quelques jours)

****Vue d'ensemble du workflow****

1. Fork → Copier le repo sur votre compte
2. Clone → Télécharger localement
3. Branch → Créer une branche pour votre travail
4. Code → Faire vos modifications
5. Commit → Enregistrer vos changements
6. Push → Envoyer sur votre fork
7. PR → Proposer vos changements au projet
8. Review → Répondre aux commentaires
9. Merge → Vos changements sont intégrés !

****Étape 1-2 : Fork et Clone****

Fork (sur GitHub) :

- Bouton "Fork" en haut à droite
- Crée une copie sur votre compte

Clone (en local) :

```
\# Cloner votre fork
git clone git@github.com:VOTRE-USERNAME/projet.git
cd projet

\# Ajouter le repo original comme "upstream"
git remote add upstream git@github.com:ORIGINAL/projet.git
```

****Étape 3 : Créer une branche****

Toujours travailler sur une branche dédiée, jamais sur main.

```
\# S'assurer d'être à jour  
git checkout main  
git pull upstream main  
  
\# Créer une nouvelle branche  
git checkout -b fix/typo-in-readme
```

Conventions de nommage :

- **fix/description** – Correction de bug
- **feature/description** – Nouvelle fonctionnalité
- **docs/description** – Documentation

****Étape 4-5 : Code et Commit****

Faire des commits atomiques – Un commit = un changement logique

```
\# Vérifier vos changements
git status
git diff

\# Ajouter et commiter
git add fichier_modifie.py
git commit -m "Fix typo in installation instructions"
```

Message de commit clair :

- Première ligne : résumé (< 50 caractères)
- Ligne vide
- Corps explicatif si nécessaire

****Étape 6-7 : Push et Pull Request****

```
\# Pousser la branche sur votre fork  
git push origin fix/typo-in-readme
```

Sur GitHub :

1. Allez sur votre fork
2. Cliquez "Compare & pull request"
3. Remplissez le template
4. Expliquez vos changements
5. Soumettez !

****Étape 8-9 : Review et Merge****

Ce qui peut arriver :

Feedback positif

- "LGTM" (Looks Good To Me)
- Merge rapide
- Remerciements

Demandes de changements

- Suggestions de style
- Demande de tests
- Questions sur l'approche
- Refus (avec explication)

Patience : Les mainteneurs sont souvent bénévoles.

****Garder son fork à jour****

Votre fork diverge du projet original au fil du temps.

Synchroniser régulièrement :

```
\# Récupérer les changements upstream  
git fetch upstream
```

```
\# Mettre à jour votre branche main  
git checkout main  
git merge upstream/main
```

```
\# Pousser sur votre fork  
git push origin main
```

Avant de créer une nouvelle branche, synchronisez toujours !

■ La séance 7 couvrira le rebase et la gestion des conflits.

****Résumé****

****Ce qu'il faut retenir****

1. **Évaluez** le projet avant de contribuer (activité, accueil, docs)
2. **Commencez petit** : typos, docs, good first issues
3. **Lisez** CONTRIBUTING.md et CODE_OF_CONDUCT.md
4. **Workflow** : Fork → Branch → Code → PR
5. **Patience** et **respect** envers les mainteneurs

Pour aller plus loin

- **First Contributions** :
<https://github.com/firstcontributions/first-contributions>
- **Good First Issue** : <https://goodfirstissue.dev>
- **How to Contribute to Open Source** (GitHub Guide) :
<https://opensource.guide/how-to-contribute/>
- **"Producing Open Source Software"** de Karl Fogel (chapitre sur les contributions)
- **Guide Mozilla** : <https://mozilla.github.io/open-leadership-training-series/>
- **MOOC Open Source Marsterclass**: <https://openourcemasterclass.org/>

****Pour la prochaine séance****

Séance 7 : Contribuer efficacement – Bonnes pratiques

Préparation suggérée :

- Forker un projet qui vous intéresse
- Lire son CONTRIBUTING.md
- Identifier une "good first issue"

****Questions ?****

:::

Passons au TD